Método de la Ingeniería

Contexto problemático

Los juegos tipo Sokoban requieren una gestión adecuada de estados, movimientos y progreso del jugador, sin embargo, cuando no se implementa una estructura organizada, pueden presentarse problemas como:

- Dificultad para manejar los estados del juego
- Errores en la validación de movimientos
- Falta de control sobre el historial de partidas
- Imposibilidad de gestionar logros o estadísticas del jugador

Estos problemas afectan directamente la funcionalidad del sistema y la experiencia del usuario

Problema

Se necesita desarrollar un sistema que permita gestionar correctamente la lógica del juego Sokoban, incluyendo el manejo de estados, movimientos, almacenamiento de resultados y logros, garantizando eficiencia y organización en el uso de estructuras de datos

Requerimientos funcionales

RF1 Registrar usuario

RF2 Iniciar sesión de usuario

RF3 Cargar nivel desde archivo JSON

RF4 Seleccionar nivel de juego

RF5 Iniciar tablero de juego

RF6 Mover jugador

RF7 Empujar cajas

RF8 Detectar condición de victoria

RF9 Guardar partida

RF10 Cargar partida guardada

RF11 Mostrar estadísticas del jugador

RF12 Mostrar ranking de jugadores

RF13 Contabilizar movimientos realizados

RF14 Contabilizar empujes de cajas

RF15 Medir tiempo de partida

RF16 Ejecutar solución automática del nivel

Búsqueda de información

Para abordar el problema, se investigaron diferentes conceptos teóricos y prácticos relacionados con estructuras de datos y lógica de programación

Temas investigados

- Listas enlazadas (estructura dinámica para almacenamiento)
- Árboles binarios (organización jerárquica de datos)
- Algoritmos de ordenamiento (como Bubble Sort vista en clase)
- Búsqueda binaria
- Manejo de estados en videojuegos
- Patrón MVC (Modelo-Vista-Controlador)

Fuentes consultadas

- Apuntes de clase y temas vistos en ellas
- Documentación oficial de Java
- Recursos sobre estructuras de datos
- Ejemplos prácticos de implementación de Sokoban
- Ejemplos prácticos de implementación de diferentes estructuras

Esta información permitió entender diferentes formas de resolver el problema y elegir las más adecuadas

Generación de ideas

Técnica utilizada: Lluvia de ideas

Se plantearon diferentes alternativas para resolver el problema:

1- Uso de arreglos simples:

Almacenar resultados en arrays tradicionales

2- Uso de listas enlazadas:

Manejar dinámicamente los resultados del jugador

3- Uso de árboles binarios:

Organizar logros de manera jerárquica

4- Uso de estructuras combinadas:

Integrar listas enlazadas + árboles binarios + otros algoritmos

Ideas propuestas

- Usar lista enlazada para almacenar historial de partidas
- Usar árbol binario para gestionar logros
- Implementar búsqueda binaria sobre resultados ordenados
- Integrar estructuras dentro del controlador del juego

Ideas descartadas

- Uso exclusivo de arreglos:

Se descartó porque no permite un crecimiento dinamico ni eficiente y requiere redimensionamiento constante

Ideas no descartadas

- Lista enlazada
- Árbol binario
- Uso de algoritmos de ordenamiento y búsqueda

Estas ideas permiten mayor flexibilidad, organización y eficiencia en el manejo de datos

Diseños preliminares

- **Lista enlazada**
- Nodo con referencia al siguiente, su uso es para almacenar resultados
- **Árbol binario**
- Nodo con hijo izquierdo y derecho, su uso es para los logros del jugador
- **Búsqueda binaria**
- Aplicada sobre resultados previamente ordenados

Estos diseños permiten visualizar cómo se integrarán en el sistema

Evaluacion:

Criterios definidos

1. **Eficiencia**

Qué tan rápido funciona la solución

2. **Escalabilidad**

Capacidad de crecer sin afectar rendimiento

3. **Facilidad de implementación**

Qué tan sencillo es implementarlo correctamente

4. **Organización de datos**

Qué tan bien estructura la información

Escala de evaluación

1 = Bajo

2 = Medio

3 = Alto

Alternativa	Eficiencia	Escalabilidad	Implementación	Organización	Total
Arreglos	1	1	3	1	6
Lista enlazada	2	3	2	3	10

Árbol binario	3	3	2	3	11
Combinación	3	3	2	3	11

Resultado final

Se selecciona la combinación de:

- Lista enlazada
- Árbol binario
- Algoritmos de búsqueda y ordenamiento

Porque ofrece el mejor balance entre eficiencia, organización y escalabilidad